

RETAILMART V3 • SQL

Subqueries Part 2 + CTEs — Quick Quiz

Quiz 1: What Makes a Subquery Correlated?

EASY

How can you tell at a glance that a subquery is correlated vs independent?

ANSWER: The inner query references a column from the OUTER table — usually via an alias like 'outer_e.dept_id' inside the inner WHERE. Without that reference, the inner query is independent and runs once. With it, the inner runs per outer row (logically).

Quiz 2: EXISTS vs IN — When?

MEDIUM

When should you prefer EXISTS over IN for the SAME logical question?

ANSWER: (1) When the inner query is expensive — EXISTS stops at first match. (2) For NOT EXISTS over NOT IN — handles NULLs safely. (3) When the inner query has potential duplicates — EXISTS doesn't care, IN materializes the whole list.

EXPLAIN ANALYZE confirms which strategy wins on your data.

Quiz 3: CTE vs Derived Table

MEDIUM

Is there any DIFFERENCE between a CTE and a derived table from PostgreSQL's perspective?

ANSWER: Since PostgreSQL 12, NO meaningful difference — CTEs are inlined just like derived tables. Same execution plan, same performance. Choose CTE for READABILITY (multi-step logic, reuse within a query). Choose derived table for inline one-shot pre-aggregation.

Quiz 4: What Makes This Subquery CORRELATED?

HARD

ANSWER: The inner query references `e.dept_id` from the OUTER row — that is the definition of CORRELATED: conceptually the inner query re-runs for every outer row, computing each department's OWN average. Result: 991 employees earn above their own department's average. Readable rewrite: pre-compute per-department averages in a CTE, then JOIN — same answer, one pass, easier to reason about.

Question



ANSWER

```
SELECT e.employee_id, e.salary
FROM stores.employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM stores.employees e2
    WHERE e2.dept_id = e.dept_id
);
```

Quiz 5: NOT IN vs NOT EXISTS Behavior

CRAZY

Inner query returns: [1, 2, NULL, 4]. Outer value = 3. What does NOT IN return? NOT EXISTS?

ANSWER: NOT IN returns UNKNOWN (treated as FALSE) — because of the NULL in the inner list, NOT IN can't conclusively say 3 isn't in the list. NOT EXISTS returns TRUE — it just checks if any row matches; the NULL in the inner result doesn't affect the existence check. This is why NOT EXISTS is the safer pattern.